



VISÃO COMPUTACIONAL

AULA 5



Prof. Leonardo Gomes



CONVERSA INICIAL

Nesta aula, daremos continuidade ao *pipeline* de processamento da visão computacional. Vamos analisar a importante etapa da extração de características que irá nos prover informações importantes sobre os objetos para descrevê-los e, posteriormente, analisá-los. Falaremos também um pouco sobre um importante algoritmo de classificação, chamado de K-NN, combinando estatística ao nosso processamento.

Ao final desta aula, esperamos atingir os seguintes objetivos, que serão avaliados ao longo de nossos estudos da forma indicada.

Objetivos

1. Compreender os principais tipos de características de objetos em visão computacional;
2. Aplicar métodos de reconhecimento de padrões como K-NN;
3. Aplicar treinamentos utilizando biblioteca *scikit-learn*.

TEMA 1 – EXTRAÇÃO DE CARACTERÍSTICAS – COR

A extração de características é uma das principais etapas dentro do *pipeline* de visão computacional. Essa etapa consiste em buscar informações que auxiliem o registro de informações cruciais do objeto em um conjunto de dados denominado *descriptor*, que pode ser aplicado na identificação ou classificação em uma etapa posterior. São vários os tipos de características que podem estar presentes no objeto como aspectos gerais, como cor, textura, suas dimensões etc. Neste tema, vamos iniciar a discussão sobre uma das mais simples que podemos extrair: a cor.

Apesar de ser uma característica simples de ser extraída, a cor é muito relevante para identificar objetos em diversas aplicações. Uma vez tendo sido definida a região de interesse do objeto, é possível extrair medidas estatísticas dos valores RGB como a média, mediana e moda.

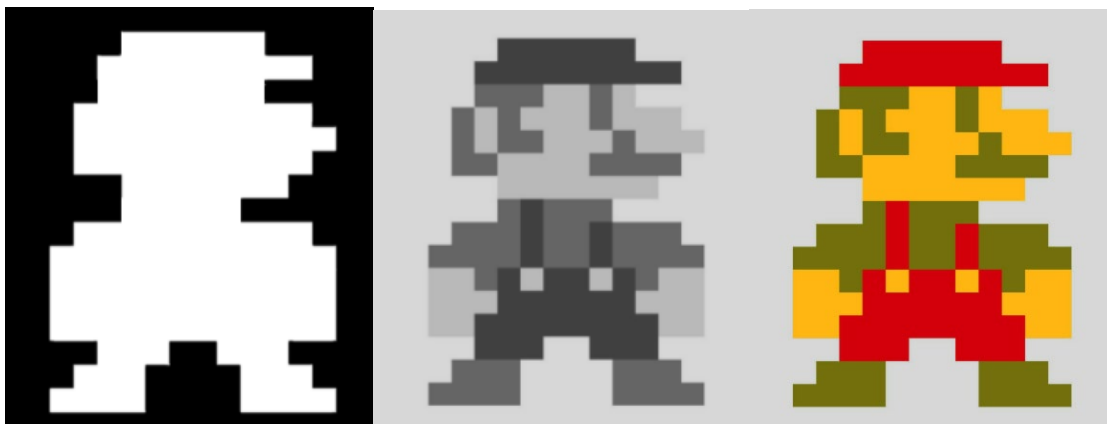
A moda, por exemplo, é o valor que aparece com mais frequência. Em uma imagem binária, por exemplo, se a moda for 255, é sinal de que a imagem possui mais pixels brancos do que preto.

No python, para obter essa informação, podemos utilizar a biblioteca *statistics* com o método *mode*: o parâmetro é uma lista de valores. As imagens

podem ser transformadas em lista de valores com o método *ravel()*. O código abaixo exemplifica essa situação.

```
import cv2
import statistics
imgCinza = cv2.imread("mario.jpg", 0)
ret, imgBin = cv2.threshold(imgCinza, 200, 255,
cv2.THRESH_BINARY_INV)
listaBin = imgBin.ravel()
listaCinza = imgCinza.ravel()
print("Binária:", statistics.mode(listaBin))
print("Cinza:", statistics.mode(listaCinza))
```

Figura 1 – Em ordem: imagem binária; tons de cinza; colorida



Fonte: Gomes, 2021.

Resultado:

- Binária: 255;
- Cinza: 215.

Como podemos ver no resultado anterior, a moda da imagem binária foi 255, indicando que a maioria dos pixels são brancos, enquanto a moda da imagem cinza foi 215, que, no caso, coincide com a cor do fundo. De forma geral, a moda é menos significativa quando trabalhamos com imagens em tons de cinza e colorida. Para essas, a média e mediana são mais interessantes, pois, caso exista uma variação da cor na imagem causada por iluminação não uniforme, esse efeito na cor é atenuado. As duas medidas podem ser obtidas



por meio dos métodos *mean()* e *median()* da biblioteca *statistics* que recebem por parâmetro uma lista de valores. Assim como no exemplo anterior, eles podem ser obtidos pelo método *ravel()*. O *openCV* também possui o método *mean()* que recebe uma imagem por parâmetro, mas não possui um equivalente para a mediana. A seguir, mais um exemplo dessa situação utilizando as imagens da Figura 1.

```
import cv2
imgCinza = cv2.imread("mario.jpg", 0)
imgCor = cv2.imread("mario.jpg")
print("Cor - Media:",cv2.mean(imgCor))
print("Cinza - Media:",cv2.mean(imgCinza))
```

Resultado:

- Cor – Media: (138.9524097442, 171.12286472320, 204.4879970550, 0.0);
- Cinza – Media: (177.39885234832, 0.0, 0.0, 0.0).

O método *mean()* retorna 4 valores por considerar imagens com 4 canais de cor. No caso da imagem colorida, vemos os valores 138, 171 e 204. Os canais de cor estão na ordem: azul, verde e vermelho. Portanto, vemos que o tom de cor em maior incidência na imagem é o vermelho.

TEMA 2 – EXTRAÇÃO DE CARACTERÍSTICAS – FORMA

Além do aspecto do objeto, sua cor e textura, as suas dimensões e forma também são muito relevantes como características. Neste tema, vamos discutir algumas das mais relevantes.

2.1 Área

A área da imagem é outra medida importante e consiste no total de pixels que representam o objeto, que geralmente resulta do total de pixels brancos nas imagens binárias que segmentam a área de interesse.

Caso exista um único objeto representado na imagem, realizar esse cálculo pode ser tão simples quanto um único loop pela imagem e realizar uma contagem pixel a pixel. No entanto, se existirem diversos objetos de interesse



em uma mesma imagem, por exemplo, uma foto telescópica do céu noturno em que as diversas estrelas são objetos de interesse, calcular a área de cada objeto individualmente requer estratégia mais elaborada. No openCV, uma forma simples de realizar esse cálculo de área é pelo método *findContours*. Esse método gera uma lista que contém o contorno dos objetos segmentados de uma imagem binária; com esses contornos, é possível extrair características com relação à dimensão de objeto na imagem.

O método *findContours* necessita de três parâmetros: a imagem binária; o modo, que indica a estratégia de armazenamento dos pontos do contorno, geralmente `cv2.RETR_TREE` é utilizado e significa que os pontos serão todos armazenados em uma estrutura de árvore de dados; e a estratégia pela qual os pontos serão detectados. Uma opção interessante nesse momento é `cv2.CHAIN_APPROX_SIMPLE`, que indica que apenas os vértices finais de cada reta serão armazenados.

Uma vez com os objetos em uma lista, é possível aplicar o método *countourArea* que recebe a imagem e devolve a quantidade de pixels daquele objeto. Um exemplo desse código é apresentado na seção seguinte junto ao perímetro.

2.2 Perímetro

Assim como a área, o perímetro, o tamanho do contorno do objeto, é outra característica importante em diversas aplicações. Uma estratégia direta para se obter esse contorno seria pela aplicação do método de detecção de bordas de Canny e, em seguida, por meio de um loop na imagem contabilizar os pixels brancos. No entanto, se tivermos mais de um objeto de interesse na imagem, essa estratégia sozinha não funciona. Podemos, no entanto, utilizar o método *findContours*, já apresentado para obter o contorno dos objetos de interesse e, na sequência, o método do openCV *arcLength* que recebe o objeto por parâmetro e retorna o seu perímetro. Confira o exemplo de código a seguir e o respectivo resultado.

```
import cv2
img = cv2.imread("mario.jpg", 0)
ret, imgBin = cv2.threshold(img, 127, 255,
cv2.THRESH_BINARY)
```



```
modo = cv2.RETR_TREE
metodo = cv2.CHAIN_APPROX_SIMPLE
contornos, hierarquia = cv2.findContours(
imgBin, modo, metodo)
objeto = contornos[0]
area = cv2.contourArea(objeto)
perimetro = cv2.arcLength(objeto, True)
print(area,perimetro)
```

Resultado:

- 1046529.0 4092.0

Tanto a área quanto o perímetro não apresentam valores que refletem com total exatidão a realidade por pequenos erros nas etapas de binarização e do método *findContours*. No entanto, apresentam resultados próximos o bastante para atender grande parte das aplicações.

2.3 Momentos Hu

Na estatística e na física, temos um conceito chamado *momentos*, que são medidas que utilizamos para descrever características de um conjunto de dados ou grandezas. Por exemplo, temos a média, desvio-padrão, assimetria, curtose, entre outros. Foge ao escopo desta aula discutir em detalhes a definição e função de cada um desses momentos. Mas o importante aqui é analisar a imagem como um conjunto de dados, em que cada pixel possui um valor de intensidade é um dado distinto, e extrair esses momentos como características para nossos descritores.

No openCV, o método *moments* recebe como parâmetro uma imagem e retorna uma lista com 24 momentos.

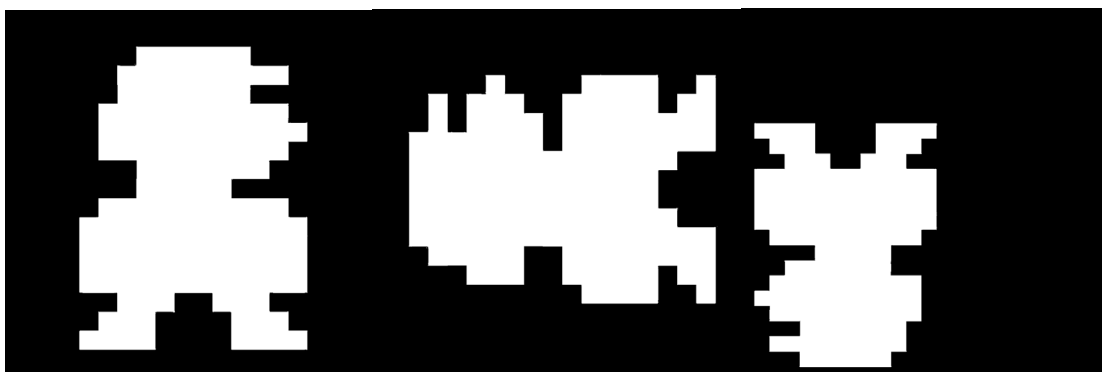
Nos anos 1960, um pesquisador chamado Ming-Kuei Hu desenvolveu equações com base nesses 24 momentos para outro conjunto de 7 valores denominados momentos invariantes de Hu. Esses valores são amplamente adotados hoje para descreverem objetos em imagens binárias de forma invariante a rotação, translação e escala. Por exemplo, suponha duas imagens distintas de uma mão fazendo um mesmo gesto. Por mais que as imagens



estejam rotacionadas entre si, em posições distintas e tamanhos diferentes, uma imagem com maior *zoom* do que a outra, os valores dos 7 momentos invariantes de Hu serão semelhantes entre si apenas por conta de o formato geral também ser semelhante. Por conta dessa propriedade, os momentos de Hu são muito úteis para descrever esse tipo de imagem. No openCV, o método *HuMoments* pode ser adotado para chegar nesses valores. Para refinar ainda mais os momentos invariantes de Hu, pensando em figuras de aspecto plano, é possível aplicar uma função logarítmica. O código a seguir exemplifica isso.

```
import cv2
import numpy as np
img = cv2.imread("mario.jpg", 0)
ret, imgBin = cv2.threshold(img, 200, 255,
cv2.THRESH_BINARY_INV)
momentos = cv2.moments(imgBin)
momentosHu = cv2.HuMoments(momentos)
print(-np.sign(momentosHu) *
np.log10(np.abs(momentosHu)))
```

Figura 2 – Imagens do mesmo objeto após passar por rotações, translações e escala



Fonte: Gomes, 2021.



Tabela 1 – Momentos invariantes de Hu, com função logarítmica das três figuras anteriores; cada linha representa um dos 7 momentos e cada coluna é um dos objetos

Momento Invariante	Primeira imagem	Segunda imagem	Terceira imagem
M1	310.578.782	310.579.137	310.641.256
M2	715.030.595	71.503.195	715.192.454
M3	1.046.807.047	104.679.578	1.047.340.884
M4	1.222.810.081	1.222.890.356	1.223.950.485
M5	-2.358.650.722	-2.358.770.759	-2.360.988.963
M6	1.581.117.681	1.581.201.138	1.582.530.855
M7	-2.424.284.738	-2.424.291.783	-2.419.932.221

Ao observarmos a tabela em questão, vemos que os valores são todos muito próximos, permitindo classificar uma elevada probabilidade de se tratar do mesmo objeto nas três imagens.

TEMA 3 – ALGORITMO K-NN

Após capturar e tratar a imagem, segmentar o objeto de interesse e extrair suas características, a etapa final do processo de visão computacional costumeiramente é a análise de reconhecimento de padrões para classificação daquele objeto. Chamamos de classe o conjunto de características definidoras de um objeto. Por exemplo, um sistema classificador de gestos manuais poderia classificar o gesto de OK pela imagem de uma mão com o dedo polegar estendido para cima, enquanto os demais dedos estão fechados na mão. Neste caso, o “gesto de OK” é uma classe na qual diferentes imagens poderiam ser classificadas.

O reconhecimento de padrões é um campo de estudos por si só que poderia ser aprofundado em diversas disciplinas. Dentro desse campo de estudo, chamamos de classificadores os algoritmos responsáveis por



classificar dados. Por se tratar de um problema muito amplo, as opções de algoritmos são várias, por exemplo: Árvore de decisão, regressão logística, redes neurais artificiais, **K-NN (K-Nearest Neighbor)**, dentre outros tantos. Cada algoritmo possui suas características como desempenho, precisão ou alguma particular afinidade com determinados tipos de problema. Portanto, escolher o melhor algoritmo cabe à análise do desenvolvedor para cada situação em particular, que acaba muitas vezes não escapando de realizar testes com diferentes estratégias antes de escolher uma.

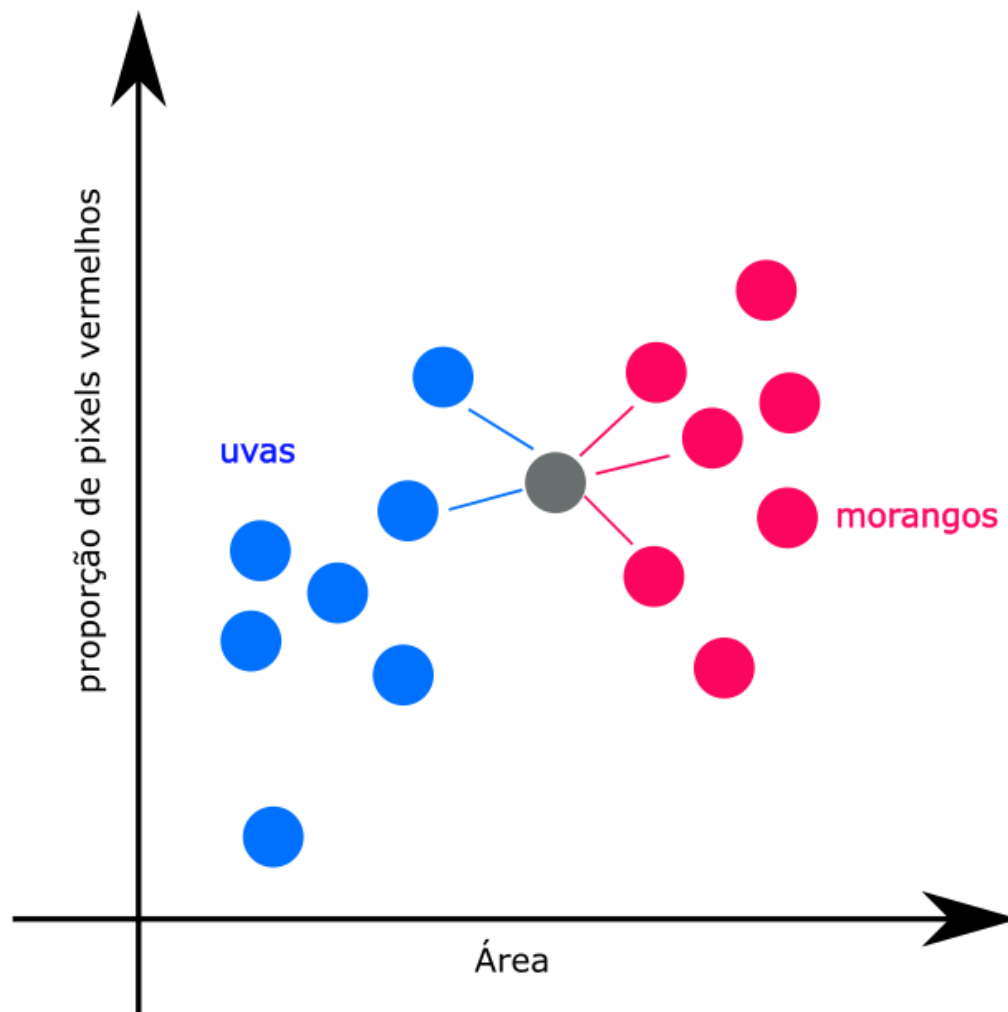
Os classificadores precisam ser treinados para reconhecer padrões, se o seu sistema irá reconhecer gestos manuais, expressões faciais, letras ou o que for ele primeiro precisa ser exposto a uma base de dados que apresenta diversos exemplos e suas respectivas classes. Esse aprendizado pode ser feito com interferência externa. Nesse caso, as imagens já vêm previamente classificadas por alguém em um banco de treinamento. Chamamos isso de **aprendizagem supervisionada**; ou, se o sistema for capaz de realizar o aprendizado sozinho e definir as próprias classes, ele é chamado de **aprendizagem não supervisionada**.

Neste Tema, vamos discutir um dos mais importantes classificadores: o algoritmo chamado *K-Nearest Neighbors* K-NN; em português, os K vizinhos mais próximos.

O K-NN é um algoritmo relativamente simples e consiste em agrupar objetos com características parecidas em grupos próprios. Ele é baseado em aprendizagem supervisionada, ou seja, utiliza uma base previamente classificada. Para classificar novos objetos, ele olha um grupo de K valores da base que sejam mais semelhantes, ou mais próximos e agrupa conforme a maioria. Observe o exemplo da Figura a seguir.



Figura 3 – Representação do K-NN para distinguir uvas e morangos pelas características do tamanho e proporção de pixels vermelhos



Fonte: Gomes, 2021.

Suponha que temos uma base como na figura anterior: diversas observações de uvas e morangos, analisadas pelas características da proporção de pixels vermelhos e área total. O ponto cinza é uma nova observação. Temos que agrupar em um dos dois grupos; porém, o dado está bem no meio dos dois agrupamentos. O que fazemos é analisar os K vizinhos mais próximos; neste caso, 5 vizinhos. Verificamos que 3 deles são morangos, enquanto 2 são uva. Então, nesse cenário, a nova observação será classificada como um morango. No caso, estamos utilizando apenas duas características, o que gera um gráfico de duas dimensões para facilitar a representação gráfica, mas poderíamos ter diversas características. Podemos também trabalhar com diversas classes. Poderiam ser utilizadas para classificar outras frutas no exemplo. Outro ponto importante é que o valor K geralmente é definido com um

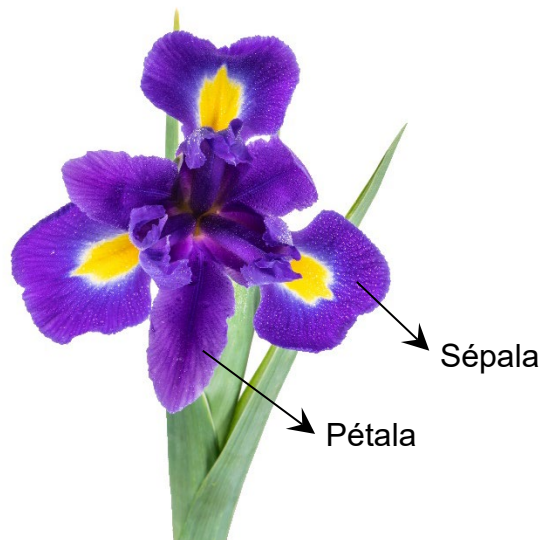


valor ímpar para minimizar a ocorrência de empates. Se ainda assim persistir, a classe que tiver o vizinho mais próximo desempata. O K-NN pode demandar muito processamento se tiver um grupo grande de dados; nesse cenário, seria possível trabalhar com uma amostra menor ou então aplicar outro método.

TEMA 4 – IMPLEMENTAÇÃO K-NN

Neste tema, vamos utilizar como exemplo uma base de dados bastante famosa: o conjunto de dados flor de Iris, que foi produzido por um biólogo chamado Ronald Fisher. Ele coletou diversas flores e as dividiu em três classes distintas: Iris-setosa, Iris-versicolor, Iris-virginica. Para cada uma, extraíram-se quatro características: comprimento e largura das pétalas, e comprimento e largura das sépalas (sépalas são folhas modificadas que ficam junto das pétalas, logo abaixo delas). A seguir, na figura e no trecho da tabela, apresentamos alguns dos itens.

Figura 4 – Flor Iris-setosa ilustrando a distinção entre sépala e pétala



Sépala-comprimento	Sépala-largura	Pétala-comprimento	Pétala-largura	Classe
5,1	3,5	1,4	0,2	Iris-setosa
4,9	3	1,4	0,2	Iris-setosa
7	3,2	4,7	1,4	Iris-versicolor
6,4	3,2	4,5	1,5	Iris-versicolor
6,3	3,3	6	2,5	Iris-virginica
5,8	2,7	5,1	1,9	Iris-virginica

Fonte: Gomes, 2021.



Com essa base de dados, é possível treinar um o algoritmo do K-NN para classificar novas flores de Íris em algumas das três classes. Para cada nova flor que desejamos classificar, temos que extrair as quatro características, o que pode ser obtido manualmente ou por meio de métodos de segmentação e extração de características como os vistos anteriormente, e verificar quais são os K vizinhos mais próximos para decidir que classe pertence.

No python, o K-NN pode ser facilmente aplicado com um pequeno conjunto de comandos da biblioteca *scikit-learn*. É necessário, no entanto, passar por algumas etapas, sendo a primeira delas criar uma base de treino com a classificação e dados de cada um dos pontos que serão analisados. Para essa base de dados, vamos carregar uma tabela excel com os dados das flores de Íris. Para isso, utilizaremos a biblioteca **Pandas**, uma biblioteca muito importante na análise de dados, que é capaz de carregar dados em diferentes formatos com uma única linha de código.

Antes de começarmos, é importante instalar os seguintes pacotes: em um terminal linux ou powershell do Windows, utilize os seguintes comandos para instalar as bibliotecas que serão utilizadas:

- Pip install openpyxl;
- Pip install sklearn;
- Pip install pandas.

O **openpyxl** permite a leitura dos dados da tabela em formatos proprietários como o excel. O **sklearn** possui os métodos de predição e análise de dados, ligados especialmente ao *machine learning*. O **pandas** é uma das principais bibliotecas de manipulação de dados. Ele opera com os dados em *dataframes* que é uma estrutura semelhante a planilhas de dados em linhas e colunas.

A primeira etapa consiste em carregar os dados e dividir em duas listas: **características** e **classificações**. As características, no caso, formam uma lista contendo outras listas com quatro valores cada para as medidas das pétalas e sépalas. O comando *pd.read_excel*, conforme vemos no código a seguir, recebe o endereço do arquivo que será carregado, podendo ser um endereço de um arquivo na internet ou local. Na sequência, podemos optar por receber o parâmetro que indique o nome das colunas da tabela de dados que está sendo carregada, conforme também é feito em seguida.



O comando `dados.head()` irá retornar os primeiros cinco itens do topo do *dataframe* chamado `dados`, sendo ideal para visualizar se os dados estão adequadamente carregados. Na sequência, para tirar os dados do formato *dataframe* e acessar no formato de lista, utilizamos o comandos: `dados.iloc[:, :-1].values`. O comando `iloc` localiza os dados que desejamos selecionar. Na nossa tabela anterior, vemos que a última coluna possui as classes e as demais as características. Por isso, para características, usamos `[:, :-1]`, que significa todas as linhas e todas as colunas menos a última, enquanto que, para as classificações `[:, 4]`, que indicam todas as linhas e **apenas** a coluna 4, o comando `values` é que efetivamente transforma de *dataframe* para lista.

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
url='https://github.com/leonhgomes/Pandas/raw/main/iris.xlsx'
nomes=['sepala-comprimento','sepala-largura','petala-
comprimento','petala-largura','Classe']
dados = pd.read_excel(url,names=nomes)
print(dados.head())
caracteristicas = dados.iloc[:, :-1].values
classificacoes = dados.iloc[:, 4].values
```

Na sequência, o comando que cria o classificador é o *KNeighborsClassifier*. Ele recebe um parâmetro que é a quantidade de vizinhos que serão utilizados no algoritmo do K-NN. No caso, serão adotados três, por se tratar de uma base pequena de dados e um valor ímpar dificulta empates.

```
knn = KNeighborsClassifier(3)
```

Na sequência, realizaremos o treinamento com os dados coletados. O comando `fit` recebe as características e as classes. Ambas as listas devem possuir a mesma quantidade de elementos e estarem na ordem equivalente para que sejam associadas adequadamente.

```
knn.fit(caracteristicas, classificacoes)
```

Na sequência, o comando `score` pode ser utilizado para avaliar a base de dados, valores próximos a 0.8, ou seja, acerto de 80% na base de treinamento e indicam um grau de precisão interessante para a maioria dos cenários. Quanto ao grau de precisão, existem dois possíveis problemas que



podemos verificar o *underfitting*, que ocorre quando a precisão do modelo é baixa desde o treinamento, o que significa que o sistema simplesmente não consegue identificar padrões. Nessa situação, é aconselhado buscar outra base de treino e o *overfitting* que ocorre quando o sistema retorna um resultado excelente para a base de treinamento. No entanto, em vez de aprender características gerais da classe, ela decora a base de treinamento e não consegue generalizar para predições fora da base.

```
print(knn.score(caracteristicas, classificacoes))
```

Por fim, para classificar um novo objeto, temos o método *predict*. Supondo que uma nova flor tenha sido observada e suas características de tamanho das sépalas e pétalas são conforme os quatro valores do código a seguir, o comando *predict* vai retornar qual a provável classe.

```
objetoNovo = ([[5, 3, 1, 0.5]])  
print(knn.predict(objetoNovo))
```

O código completo é apresentado a seguir.

```
import pandas as pd  
from sklearn.neighbors import KNeighborsClassifier  
url='https://github.com/leonhgomes/Pandas/raw/main/iris.xlsx'  
nomes=['sepala-comprimento','sepala-largura', 'petala-  
comprimento','petala-largura','Classe']  
dados = pd.read_excel(url,names=nomes)  
#print(dados.head())  
caracteristicas = dados.iloc[:, :-1].values  
classificacoes = dados.iloc[:, 4].values  
knn = KNeighborsClassifier(3)  
knn.fit(caracteristicas, classificacoes)  
print(knn.score(caracteristicas, classificacoes))  
objetoNovo = ([[5, 3, 1, 0.5]])  
print(knn.predict(objetoNovo))
```

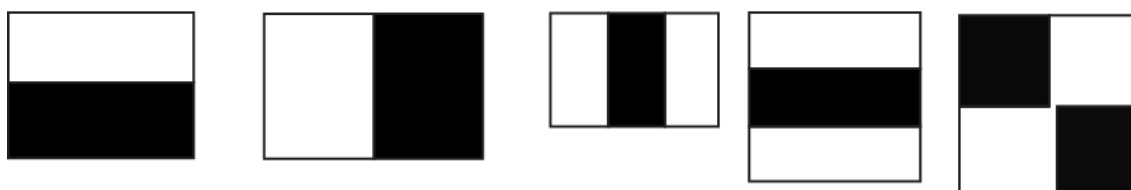
No código anterior, obtivemos um score para o modelo de treinamento de 0.96 e a predição foi de o objeto novo pertencer a classe Iris-setosa.

Um algoritmo de grande importância para detecção de objetos, amplamente adotado especialmente no contexto de faces, é o algoritmo *Haar cascade*, também conhecido por Viola-Jones, por conta dos nomes dos autores Paul Viola e Michael Jones que criaram o algoritmo em 2001.

O algoritmo consiste em filtros convolucionais para detecção de características, uma tarefa com relativamente pouco processamento envolvido, o que garante um excelente desempenho mesmo em aplicações tempo-real, até hoje suas implementações são adotadas amplamente em soluções comerciais para detecção de faces, como câmeras e marcações em redes sociais. No entanto, o mesmo algoritmo com diferentes bases de treino e configurações é capaz de ser programado para detectar os mais distintos objetos.

A estratégia consiste em buscar pontos característicos na imagem. Esses pontos característicos são obtidos através de uma série de máscaras convolucionais que passam pela imagem gerando valores mais altos se a região se assemelha com a imagem e valores mais baixos caso contrário. A Figura a seguir exemplifica algumas dessas máscaras.

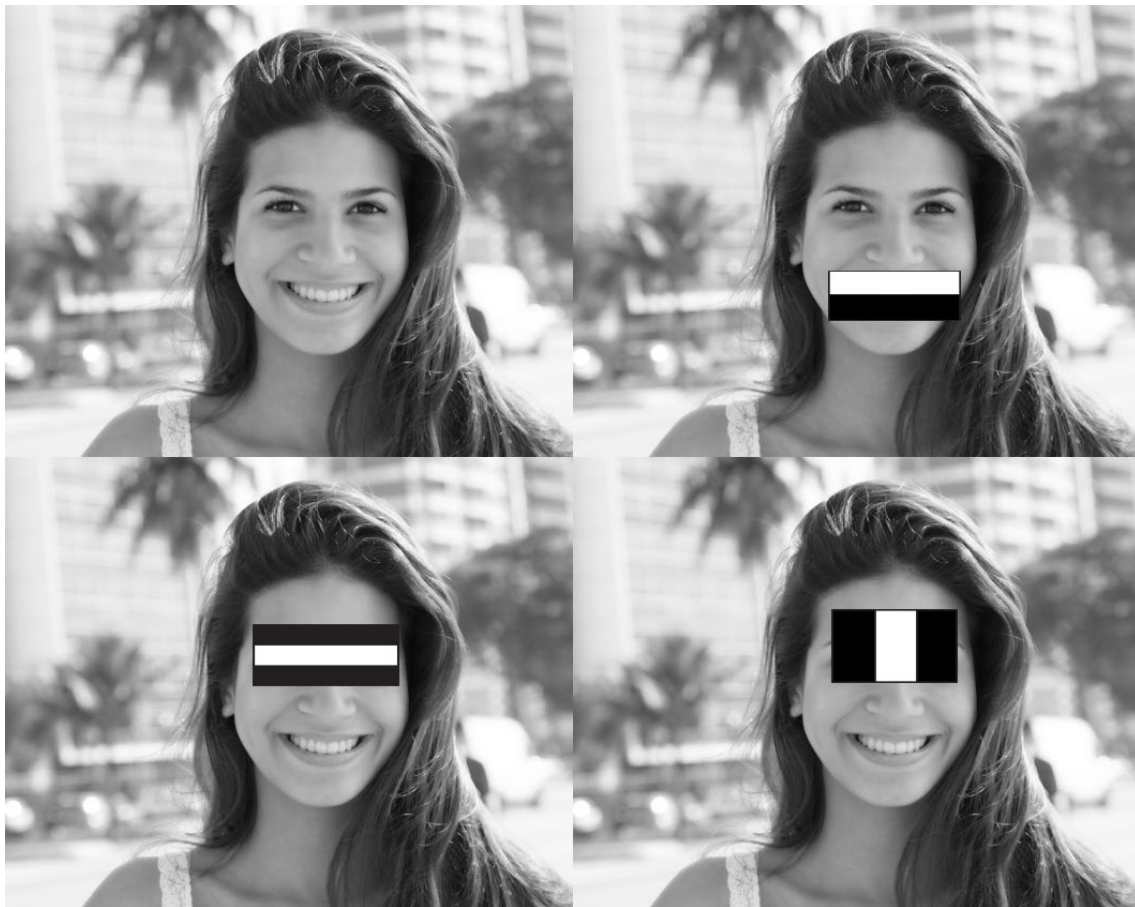
Figura 5 – Representação das máscaras de características Haar. As duas primeiras são para bordas horizontais e verticais; as duas seguintes são para detecção de linhas verticais e horizontais; e, por fim, máscara de quatro retângulos



Suponha a primeira imagem na figura anterior. Ela descreve uma região de pixels mais claros sobre uma região de pixels escuros relativos entre si. As áreas na imagem que tenham esse tipo de característica terão um valor mais alto. Se esse valor for acima de determinado limiar, serão registradas como um ponto característico. Os diversos filtros são testados e todas as regiões características são marcadas.

Na sequência, conjuntos desses pontos característicos podem ser utilizados para determinar as regiões de interesse. Primeiro, todos os pontos característicos encontrados na imagem têm posições relativas registradas. Na sequência, essas informações são comparadas com uma base de treinamento e, se determinado conjunto de padrões de características existir em uma área qualquer da imagem aquela área, será marcada como área de interesse. Na figura a seguir, veja os padrões de Haar comumente encontrados em um rosto.

Figura 6 – Regiões com características Haar



Créditos: Daniel M Ernst/Shutterstock.

Para o treinamento da base de Haar, geralmente é feito com um conjunto muito grande de imagens ditas positivas e negativas. Positivas são aquelas imagens que possuem objeto de interesse e negativas que não tem; assim, o algoritmo aprende a interpretar o que é e o que não é região de interesse. Vale reforçar que, apesar do uso de técnicas de *machine learning*, o algoritmo de *Haar cascade* é feito para detecção de objetos que internamente adota classificadores como uma etapa, mas ele não é um classificador em si.



No OpenCV *Haar cascade*, também é implementado e pode ser facilmente aplicado com poucas linhas de código. A primeira coisa que precisamos é de uma base de treino para o tipo de objeto que desejamos identificar. O arquivo geralmente é descrito em forma de arquivo .xml. Objetos comuns, como faces, olhos, carros e placas de automóveis, possuem bases de treinamento prontas que são facilmente encontradas online. Mas é possível criar a própria para identificar outros tipos de objetos.

A seguir, temos o código completo que identifica rostos em uma foto e, dentro da área dos rostos, busca por olhos. Faces e olhos, cada um possui um classificador com um arquivo diferente. No código a seguir, está também o link de onde os arquivos utilizados podem ser encontrados.

O método *CascadeClassifier* é utilizado para carregar o arquivo *Haar cascade* do objeto que desejamos encontrar. Já o método *detectMultiScale* efetivamente faz a busca e retorna uma lista de retângulos. O método aceita diversos parâmetros, permitindo personalizar o algoritmo para diversas situações. No caso do código a seguir, o primeiro parâmetro é a imagem; o segundo parâmetro é o *fator de escala*, que indica o quanto a imagem deve ser reduzida para a análise. Suponha que o treinamento tenha sido feito com imagem de tamanho 500x500 pixels. Então, o algoritmo tende a performar melhor se receber imagens de tamanho semelhante. Portanto, pode ser interessante aumentar ou diminuir a imagem para melhorar os resultados. O terceiro parâmetro é a quantidade de vizinhos cada retângulo de resposta deve ter, em termos simples, quanto maior o valor menos rostos serão encontrados; porém, também diminui falsos positivos.

Para desenhar os retângulos, utilizamos o método *cv2.rectangle*, que recebe por parâmetro a coordenada dos dois pontos extremos do retângulo, os pontos de coordenadas com menor valor e os de maior valor, depois a cor do retângulo (na ordem azul verde, vermelho) e, por fim, quantos pixels de espessura.

```
import cv2
# "https://raw.githubusercontent.com/leonhgomes/Pandas/main/Haarcascade_frontalface_default.xml"
face_cascade=
cv2.CascadeClassifier('Haarcascade_frontalface_default.xml')
```

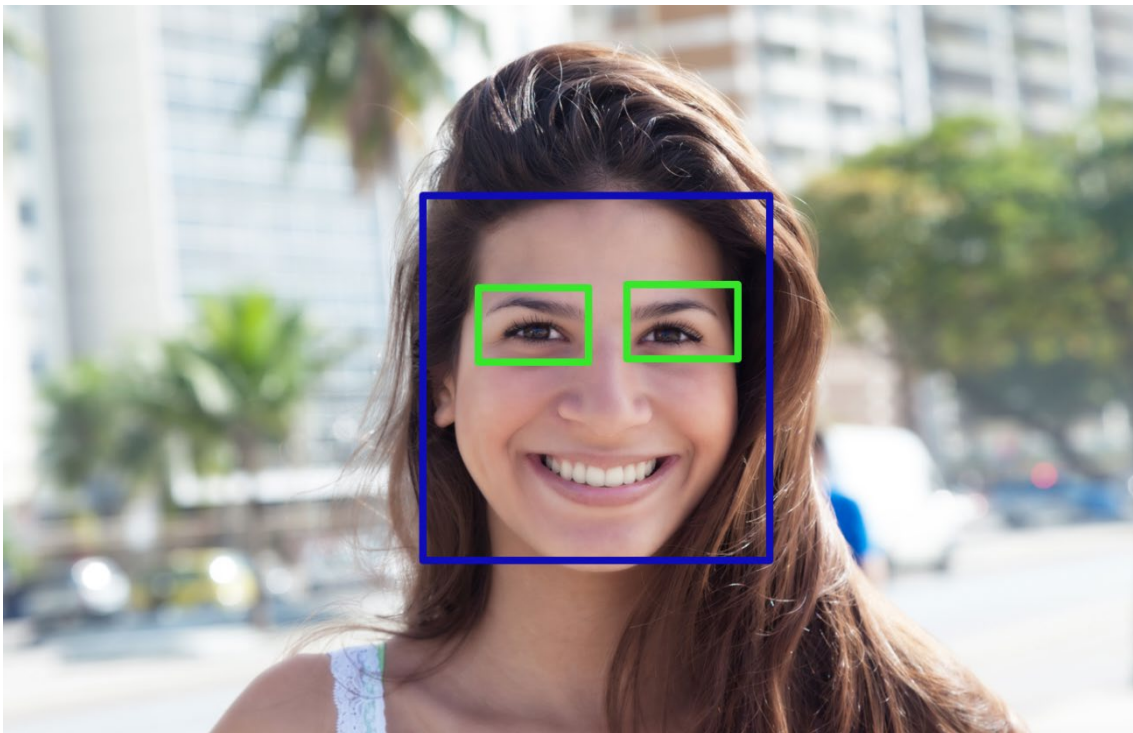


```
#"https://raw.githubusercontent.com/leonhgomes/Pandas/main/Haarcascade_eye.xml"
olho_cascade= cv2.CascadeClassifier('Haarcascade_eye.xml')
img= cv2.imread("rosto.jpg")
cinza= cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces= face_cascade.detectMultiScale(cinza, 1.3, 1)
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
    area_cinza = cinza[y:y+h, x:x+w]
    area_cor = img[y:y+h, x:x+w]

    olhos = olho_cascade.detectMultiScale(area_cinza,1.3,5)
    for (ex, ey, ew, eh) in olhos:
        cv2.rectangle(area_cor, (ex, ey), (ex+ew, ey+eh), (0,
255, 0), 2)
cv2.imshow('img', img)
cv2.waitKey(0)
```

Resultado:

Figura 7 – Face e olhos detectados com algoritmo *Haar cascade*



Créditos: Daniel M Ernst/Shutterstock.



FINALIZANDO

Nesta aula, vimos com mais detalhes duas das principais etapas do *pipeline* de visão computacional, a extração de características e, na sequência, estudamos um importante método de reconhecimento de padrões, o K-NN. Por fim, vimos também outro algoritmo de muita importância na detecção de objetos, o *Haar cascade*, que combina os conceitos de extração de características e reconhecimento de padrões para retornar às regiões de interesse de uma imagem. Com esses conceitos, esperamos que seja possível ao estudante de visão computacional começar a vislumbrar possíveis passos para solucionar problemas do mundo real com auxílio dessas técnicas apresentadas até aqui. Futuramente, veremos alguns problemas e discutiremos suas possíveis soluções.



REFERÊNCIAS

- ANDALÓ, F. **Modelagem e Animação 2D e 3D para Jogos**. São Paulo: Érica, 2015.
- ARRUDA, E. P. **Fundamentos para o Desenvolvimento de Jogos Digitais**: Série Tekne. São Paulo: Bookman, 2014.
- BANIN, S. **Python 3**: conceitos e aplicações: uma abordagem didática. São Paulo: Érica, 2018.
- BARELLI, F. **Introdução à visão computacional**. São Paulo: Casa do Código, 2019.
- CHONG, A. **Animação Digital** – Coleção Animação Básica. São Paulo: AMGH, 2014.
- FRIGERI, S. **Computação Gráfica**. 1. ed. São Paulo: SAGAH, 2018.
- GONZALEZ, R. C.; WOODS, R. E. **Processamento Digital de Imagens**. 3. ed. São Paulo: Prentice Hall, 2010.
- SANTOS, F. dos; FERREIRA, S. F. **Geometria Analítica**. ArtMed, 2009.
- WINTERLE, P. **Geometria Analítica**. 2. ed. São Paulo: Pearson, 2014.